

Product Vision and Core Objectives

Modern error monitoring tools (e.g. Sentry, LogRocket) already capture rich data like errors, logs, and user sessions to speed up debugging ¹ ². Our AI-Powered Browser Extension Bug Intelligence System goes further: it's a full-scale platform (not just a simple extension) that **automatically captures everything** about an issue, **reconstructs the user's session**, **uses AI to diagnose root causes**, and **suggests fixes**. The goal is to help developers find and fix bugs with minimal manual effort. In practice, this means the system must capture all relevant data (errors, logs, network calls, DOM changes, console output, screenshots, etc.), organize it by user session, analyze it with AI, and produce actionable summaries and recommendations. The ultimate objective is to "capture truth" from the browser, "understand truth" via intelligence, and "explain truth" in a clear report.

Key objectives include:

- **Capture everything:** Track all errors, logs, HTTP requests/responses, user interactions (clicks, inputs, navigation), and even screenshots or recordings at the time of the bug ¹ ³. This may involve using content scripts to intercept network calls and console/error events, and recording user events via a library like *rrweb* ³.
- **Reconstruct sessions:** Bundle all events into a timeline per user session. Generate a unique `sessionId` on page load (e.g. via `crypto.randomUUID()`) and send batched events to the backend in chronological order. This session context lets us replay and analyze what the user did before the failure.
- **Understand issues with AI:** Feed the error event, its stack trace, and the related session timeline into an LLM (e.g. GPT-4 series) to extract key information: a concise **bug summary**, the **root cause hypothesis**, **reproduction steps**, and **fix suggestions** ⁴. The AI engine can also assign a severity level.
- **Assist in fixing:** Automatically integrate with issue trackers (Jira, GitHub, etc.) by creating rich tickets containing logs, summaries, screenshots, and replay links. Prioritize issues based on impact (e.g. number of users affected, error frequency, or HTTP status codes) to ensure critical problems are addressed first.
- **Minimize user effort:** Developers should only need to install the extension and optionally trigger a report; the system should do the rest. Smart defaults (automatic error capture, optional consent for deeper data) and guided interfaces (report bug buttons, AI chat assistant) make the workflow seamless.

By comparing to existing platforms, we ensure our MVP covers core features: error logging, session traces, and basic analysis. For example, LogRocket "captures errors, performance issues, real user interactions, and more" in one platform ¹. We similarly plan to bundle logging, session replay, and AI analysis into a unified debug platform.

Inventory of Existing Tools and MVP Features

Before building, we review related tools to prioritize our MVP:

- **Error & Log Aggregators:** Tools like Sentry, Bugsnag, Rollbar focus on capturing stack traces and logs. They group and alert on errors to help developers triage. However, they often lack rich

session context. We'll adopt their best practices for error grouping, alerting, and stack trace parsing.

- **Session Replay Platforms:** Services like LogRocket and PostHog record user sessions (DOM snapshots, events, console logs). LogRocket explicitly provides “high-fidelity session replay” along with error context ⁵. Our MVP should support basic DOM/event capture (via *rrweb* or similar) to let developers see exactly what the user saw.
- **Performance Monitoring:** While beyond core MVP, tools like Datadog or New Relic correlate errors with performance metrics. We won't include metrics in MVP, but we'll design extensible events so later we could add things like CPU usage or load times.
- **AI Analysis:** Early AI tools (e.g. the open-source *Bugger*) show it's feasible to use GPT to analyze bug descriptions and suggest fixes ⁴. An MVP might start with a simple prompt that takes an error message and a sequence of events, then asks GPT for a summary and steps. Over time, we'll refine prompts and pipelines.

MVP Feature Prioritization: Based on impact vs. effort, Phase 1 MVP should include: automatic error and console logging, HTTP request/response capture, event tracking (clicks/inputs), batching of events, and backend ingestion for storing these events. We'll provide a simple UI (browser popup) to view reports and optionally create tickets. AI-driven analysis and session replay can come in later phases once core data is flowing.

High-Level Architecture

The system consists of four main layers:

- **Data Capture Layer (Browser Extension):** A Chrome/Firefox extension that injects scripts to monitor the web app in real time. It captures **content scripts** for user interactions (clicks, keystrokes), **error tracking scripts** for JS errors, **network interception** for XHR/fetch, and **console logging**. A background service worker in the extension batches these events (e.g. `if (events.length>20 || time>5s) sendBatch()`) and sends them to the backend via a REST API. We'll use a modern frontend stack (React + TypeScript, with Vite or TailwindCSS) for any extension UI like bug-report popups or side panels. This layer must be lightweight – e.g., relying on incremental DOM capture (via *rrweb*) instead of expensive operations ³.
- **Backend API & Processing (NestJS):** A NestJS server provides endpoints like `POST /events/batch`. Ingestion flows into a Redis-backed queue (e.g. via BullMQ). Worker processes consume from the queue to process events: persist them in PostgreSQL, compute any immediate analytics, and feed data to AI jobs. Using BullMQ (Redis queue) decouples the real-time capture from heavy processing. As one source notes, “NestJS + BullMQ with Redis ... lets tasks run in the background, freeing up the API to respond instantly” ⁶ ⁷. The backend modules will include session management, event storage, error detection, analysis pipelines, and optional ticket integration.
- **Data Storage Layer:** The main database is PostgreSQL with the **pgvector** extension for vector embeddings. We'll store sessions, raw events, and error summaries in normalized tables (see schema below). Redis serves both as the queue for processing (via BullMQ) and as a cache (e.g. quick lookup of active sessions). For any media (screenshots, recordings), we'll use object storage (AWS S3 or Clouinary).
- **AI/Intelligence Layer:** This consists of services that invoke LLMs and embedding models. We'll use OpenAI's models (e.g. GPT-4, embedding endpoints) via the OpenAI SDK. For flexibility, we

may route calls through **OpenRouter**, which provides a unified API to multiple LLM providers ⁸. However, note that OpenRouter is optimized for developer speed and prototyping; for production needs we'd ensure we have direct access or an enterprise gateway with compliance if needed. Our AI pipeline will process inputs (error message, stack trace, recent session events, API logs) through structured prompts to generate outputs like a bug **summary**, **root cause**, **reproduction steps**, and **fix suggestion**. It will also create or update an internal knowledge base of bug clusters by storing error embeddings in pgvector for similarity search ⁹.

Visually, the data flow is:

```
Browser Extension (inject scripts)
  ↓ collects events and logs
  → sends batches to
Backend API (NestJS controllers)
  → enqueues jobs in Redis (BullMQ)
  → worker processes store data in PostgreSQL, generate embeddings, run AI
  analysis
  → results (summaries, etc.) sent to UI or ticket systems
```

This layered approach scales and separates concerns. The use of event streams makes it similar to modern observability tools (e.g. PostHog's session recordings ³).

Browser Extension (Data Capture Layer)

- **Content Scripts:** Injected into pages to monitor the app. They will hook into global error handlers (`window.onerror`), override `fetch` / `XMLHttpRequest` to log API calls, and listen to user events (clicks, form inputs). A library like *rrweb* can capture DOM snapshots and user interactions efficiently ³. Every event is structured with a schema (see below).
- **Error & Console Tracking:** We log uncaught JS errors, along with the message, stack trace, file, and line number. Console output (warn/error) can also be captured. For HTTP calls, both request and response (status, body) are recorded.
- **Event Schema:** Each captured event follows a base structure, e.g.:

```
{
  "id": "uuid",
  "sessionId": "...",
  "timestamp": 162...,
  "type": "api_response",
  "url": "https://api.example.com",
  "payload": { "status": 500, "method": "POST", ... }
}
```

This matches the design where everything is an event with a `type` (click, input, navigation, api_request/response, error, console, etc.) ⁴ 🔥. Using JSON payloads allows flexible data storage.

- **Background Service Worker:** Runs continuously to buffer events. It periodically flushes events to the backend. For example, if N events have accumulated or T seconds have passed, it will `POST /events/batch` with the array of events. This batching improves efficiency (fewer

network calls) while ensuring no data is lost on navigation or failure. The workflow is:

```
[Content script] → [Browser local buffer] → (batch every few seconds) →  
[Background script] → (HTTP) → [Backend API]
```

- **Extension UI:** A React-based popup or side-panel lets users manually report a bug or view details of the current session's events. This UI can also show real-time status (e.g. "Events collected, ready to upload") and allow optional screenshot capture. We'll use TypeScript, Vite, and TailwindCSS for this UI. State management (e.g. with Zustand) can hold the session's metadata client-side before sending.

Backend Ingestion and Processing (NestJS)

- **API Endpoint:** The extension will call `POST /events/batch` with a JSON body of event objects. The NestJS controller authenticates (if needed), then quickly enqueues the data for processing.
- **Event Queue (BullMQ):** We use BullMQ for Redis-backed job queues. As the NestJS docs note, "BullMQ uses Redis to persist job data, enabling scalable producers/consumers across nodes" ¹⁰ ⁶. Each batch becomes a job. This lets our API respond fast (just confirming ingestion) and offloads heavy work to workers.
- **Worker Processing:** A worker pulls each job: it iterates over the events in the batch, saves them into the database (sessions/events tables) and checks for errors. If an event is an `error`, the worker logs it in a separate `errors` table and may trigger AI analysis or alerting. After storing events, a worker may also trigger background analytics (e.g. compute error embeddings or run clustering jobs).
- **Database Design:** We will have tables like:
 - `sessions`: (id PK, start_time, user_id, metadata JSON). Each browser session gets a row.
 - `events`: (id PK, session_id FK, type, timestamp, payload JSONB). Stores all raw events. Indexed by session and timestamp for timeline reconstruction.
 - `errors`: (id PK, session_id, event_id, message, stack, vector VECTOR). Stores error details and an embeddings vector (using `pgvector`).
 - Additional tables: e.g. `tickets` or `replays` linking to sessions.

Storing payloads as JSONB lets us log arbitrary data (DOM snapshots, API bodies) without rigid schema changes. For efficiency, we'll index by common filters (type, status codes, URL path) if needed.

- **Timeline Reconstruction:** To rebuild a session, we simply query `SELECT * FROM events WHERE session_id = ? ORDER BY timestamp` ⁴ 🔥. This sequence can be fed to the UI or to the session replay feature.
- **Error Correlation:** When an error occurs, the backend can fetch the last *N* events (e.g. last 10 actions) and the last API request/response to add context. This historical context is attached to the error record for AI analysis and developer viewing.

Data Schemas and Event Formats

Defining clear data formats up front ensures all components can interoperate. Key designs:

- **Event Schema (frontend → backend):** As above, a base event has `id`, `sessionId`, `timestamp`, `type`, `url`, and a `payload` object. The `type` field is an enum (`click`, `input`, `navigation`, `api_request`, `api_response`, `error`, `console`, etc.). Each type has a specific payload shape (see example JSON in the prompt). Using this schema, any event is a single row in `events` (with payload in JSONB).
- **Session Metadata:** On page load, the extension generates a new `sessionId` (UUID) and stores it in `sessionStorage`. It sends a session-start event with metadata:

```
{
  "sessionId": "...",
  "startTime": 162...,
  "userAgent": navigator.userAgent,
  "url": document.location.href,
  "userId": currentUserId
}
```

The backend inserts a `sessions` row with this info. (If the user is not logged in, `userId` may be null or a guest ID.)

- **Database Tables:** Rough SQL layouts:

```
CREATE TABLE sessions (
  id UUID PRIMARY KEY,
  start_time TIMESTAMP,
  user_id TEXT,
  metadata JSONB
);
CREATE TABLE events (
  id SERIAL PRIMARY KEY,
  session_id UUID REFERENCES sessions,
  type TEXT,
  timestamp BIGINT,
  payload JSONB
);
CREATE TABLE errors (
  id SERIAL PRIMARY KEY,
  session_id UUID,
  event_id INTEGER REFERENCES events,
  message TEXT,
  stack TEXT,
  vector VECTOR(1536) -- pgvector embedding
);
```

We store the error's text and stack for quick queries, plus a pre-computed embedding vector (e.g. 1536-dim from OpenAI) for similarity search. All heavy data (DOM, screenshots) can go in S3, referenced by URL in the payload.

- **Embeddings and Clustering:** When a new error is recorded, we generate a text embedding (via OpenAI's embedding API) of the error message and stack. This vector is saved in `errors.vector`. We can then cluster similar errors by a KNN query: `ORDER BY vector <-> query_vector` finds nearest neighbors ⁹. In practice, this lets us auto-group similar incidents. We may also maintain an `error_clusters` table that periodically re-runs clustering (e.g. K-means or HNSW) on high-volume errors.
- **Ticket Payload:** For automatic ticket creation, we define a JSON payload template:

```
{
  "summary": "Error: Cannot read property of undefined",
  "steps": ["Open app", "Click Submit", "..."],
  "logs": [ "...last events..." ],
  "screenshot": "https://s3.amazonaws.com/.../error.png",
  "replayUrl": "https://ourplatform.com/replay/SESSION_ID"
}
```

This can map to JIRA or GitHub issues.

AI & Intelligence Layer

The most advanced features revolve around AI analysis and tooling:

- **AI Bug Analysis (LLM):** We feed the LLM a carefully crafted prompt containing: the error message/stack, a summary of recent user actions (e.g. "User clicked X, then form Y, then API POST to /checkout returned 500"), and any relevant logs. The model output is a JSON with fields like `"summary"`, `"root_cause"`, `"steps_to_reproduce"`, `"fix_suggestion"`, and `"severity"`. Using structured JSON ensures machine-readability. For example, a prompt might say:

"I have an error log from a web session. The error message is 'Cannot read property of undefined' in file `app.js` at line 45. The user's last actions were: click on Submit button, then a POST /checkout request returned status 500. Based on this, provide a bug summary, root cause, reproduction steps, and fix suggestion in JSON."

Early experiments (e.g. the open-source *Bugger* tool) show that GPT-4 can robustly categorize bugs and suggest fixes ⁴. We will iterate on prompt design and possibly break down the task (e.g., first summarize, then ask for root cause, etc.) to improve accuracy. Critically, the AI has access to precise session context (unlike a developer who might lack info), so it can reason about timing and causality.

- **Error Clustering:** Each unique error (same message/stack) gets an embedding. By querying similar embeddings (PostgreSQL's `<->` operator), we can group errors into clusters of related issues ⁹. This helps identify recurring problems. For example:

```
SELECT *, vector <-> '[...current_vector...]' AS distance
FROM errors
ORDER BY distance
LIMIT 5;
```

yields the closest past errors. We might periodically run a clustering job (e.g. HNSW or K-means via pgvector) to label “error categories” so developers can see how many sessions hit the same underlying bug.

- **Session Replay (rrweb):** Using *rrweb*, the extension records DOM mutations and user events. The backend can reconstruct and serve a replay UI with a timeline scrubber ³. This allows a developer to play back exactly what the user saw. For example, on encountering an error, the user’s screen snapshot at that moment is available, and the timeline shows what they clicked or typed. As one Sentry blog noted, session replay “captures user interactions (mouse movements, clicks, scrolling, keyboard inputs) enabling developers to recreate the exact user experience” ². This visual context is invaluable for tricky bugs.
- **Screenshots & Video:** We provide hooks to capture screenshots automatically on errors (`chrome.tabs.captureVisibleTab()`) or manually via UI. For more detailed context, a background media recorder (WebRTC `MediaRecorder`) can record a short video (subject to user permission and privacy concerns). These media files are uploaded to S3 and linked in the bug report.
- **Auto Ticket Creation:** Upon confirming a bug, the system can format a ticket payload (JSON above) and call a project’s issue-tracker API. This creates a templated ticket containing the summary, steps, logs, and links to the replay and media. Such integration ensures the debugging process flow is end-to-end (from bug discovery to issue filed) with minimal manual effort.
- **AI Debug Assistant (Conversational):** As a bonus, we can offer a chat interface: “Why did this fail?” The assistant is given the session’s context and can answer follow-up questions. Technically, it’s an LLM prompt filled with context from the stored events and error, enabling interactive debugging help.
- **Severity Detection:** A rule engine assigns an initial severity (e.g. HTTP 500+ or unhandled exceptions map to “high”), which the AI can override or refine if needed.

Core System Philosophy

- **Capture Truth:** The foundation is a strict event-based model. Every user action and system output is an event. This unifies all data (clicks, API calls, DOM changes, console logs) into a single source of truth ³ ⁵. The design avoids losing any context; if it happened in the browser, we log it.
- **Understand Truth:** On top of raw data, we build intelligence. Clustering and AI analysis sift through events to identify recurring patterns and causes. The LLM “understands” the trace of the bug by reading the chronological events.
- **Explain Truth:** Finally, the platform generates human-friendly explanations. By providing concise summaries, root causes, and reproduction steps, we turn noisy logs into actionable insights.

As one guide cautions, the key is not just AI, but a **clean event architecture and session model** ⁵ ¹¹. Without consistent, structured data, the AI layer would have nothing reliable to work on. Thus, we invest heavily in robust data capture and storage, then layer intelligence on top.

Key Tech Stack

- **Extension:** React + TypeScript, built with Vite; state management (Zustand) if needed; UI styled via TailwindCSS; uses rrweb for session recording.
- **Backend:** NestJS (TypeScript); BullMQ/Redis for queues; WebSocket or HTTP for any real-time updates; OpenAPI (Swagger) for API docs.
- **Database:** PostgreSQL with pgvector extension.
- **Caching/Queue:** Redis (BullMQ) and possibly caching frequently accessed sessions or AI results.
- **Storage:** AWS S3 (or Cloudinary) for screenshots/recordings; possibly Redis cache for small images.
- **AI:** OpenAI GPT-4 series (via OpenRouter or direct), using the OpenAI SDK for both chat completions and embeddings; structured prompt pipelines (inject session JSON, ask for summary).
- **Session Replay:** rrweb with incremental snapshots; replay served via a web UI in our platform.

This stack is chosen for familiarity and synergy: NestJS + Redis is a well-known scalable architecture ⁶, and PostgreSQL/pgvector keeps all data unified. We avoid more exotic components early on to hit market faster.

Implementation Roadmap

We propose a phased delivery with milestones and QA checks at each stage:

- **Phase 1 (MVP):** Error and console logging, network (API) logging, basic event batching from extension to backend. This includes implementing the event schema, database tables (sessions, events, errors), and a working `/events/batch` endpoint. *Milestone:* We can demo capturing a JS error with stack trace and (e.g. from a sample app) and seeing it stored in the DB. **QA gate:** Validate that events are captured reliably in different browsers without significant UI lag. Ensure sensitive data (passwords, cookies) are masked.
- **Phase 2:** User action tracking (clicks, inputs, DOM navigation) and session reconstruction. Begin session replay integration (client-side rrweb and a simple replay frontend). Add screenshot-on-error. Implement ticketing: an endpoint to send bug summary to a mock ticket system (e.g. Postman Echo). *Milestone:* Demo replay of a user clicking through a form, encountering an error, and automatically generating a formatted bug report. **QA gate:** Test event ordering accuracy, replay fidelity, and media capture works across pages.
- **Phase 3:** AI integration – implement the AI analysis pipeline. Design prompts and test LLM outputs for a variety of errors. Display AI-generated summary and suggestions in the UI. Introduce severity scoring. *Milestone:* Show an LLM summary in the bug report UI (e.g. “likely NPE due to undefined user object; reproduce by logging out then clicking Submit.”). **QA gate:** Ensure AI outputs are reviewed by developers for correctness. Confirm fallback if AI fails (still have raw data).
- **Phase 4:** Advanced intelligence – error clustering and session search. Background jobs to compute embeddings for errors and allow searching for similar past bugs. Polish the AI assistant

chat. Add fine-grained prioritization (user counts, business impact metrics). *Milestone:* Demonstrate a cluster view of errors (e.g. “5 similar bugs occurred”), and an AI assistant answering a developer’s “Why did this fail?” question contextually. **QA gate:** Validate clustering accuracy (false positives), chat relevance, and overall performance.

- **Continuous:** At each phase, include load/performance testing (e.g. simulate 1000 events per minute) and ensure the system scales. Implement security reviews (especially for the extension code) and compliance checks (opt-in data tracking, privacy masking) early on. Monitor system resource usage (DB size from events, Redis load on queue, API response times) and optimize (e.g. compress event payloads, shard queues) as needed.

Each phase should deliver a minimally shippable product increment that can be tested with real developers. Regular feedback (internal demos, beta testers) will guide prioritization of features like user consent, data retention policies, or integration specifics.

Conclusion

This plan outlines a **full-scale AI debugging platform**, not just a throwaway extension or basic logger. We leverage modern architecture (event-driven design, Node.js with NestJS, Redis queues, PostgreSQL/pgvector) and AI models to create a system where capturing truth is automated and understanding truth is accelerated by intelligence ³ ⁵. Key to success is starting with a rock-solid core: capturing all relevant data in a structured way and reliably reconstructing sessions. Only then do we “add intelligence” (AI summaries, clustering) on top, ensuring the AI has a solid factual foundation.

Remember, the **clean event/session system** is the critical success factor. Performance and data volume are challenges, so we’ll need batching, compression, and smart filtering (e.g. mask sensitive fields, drop overly verbose logs) ¹². Privacy is paramount: no tracking of HTTP-only cookies or personal inputs without consent. By carefully planning each component – from the extension scripts to database schema to AI prompts – and iterating in phases, we ensure nothing is overlooked.

In the end, the result is more than a Chrome extension or error logger: it’s an *observability platform* powered by AI that helps developers get from “bug happened” to “bug fixed” faster than ever.

¹ ⁵ JavaScript error logging | LogRocket

<https://logrocket.com/for/javascript-error-logging>

² Replaying Backend Errors using Sentry’s Session Replay | Sentry Blog

<https://blog.sentry.io/replaying-backend-errors-using-sentrys-session-replay/>

³ Session replay architecture - Handbook - PostHog

<https://posthog.com/handbook/engineering/session-replay/session-replay-architecture>

⁴ GitHub - wchao1115/Bugger: Bugger helps you manage your bugs. This repo is AI-only. No manual typing. No exceptions.

<https://github.com/wchao1115/Bugger>

⁶ ⁷ ¹² Scalable Background Jobs with NestJS & Redis | by Neurobyte | Medium

<https://medium.com/@kaushalsinh73/scalable-background-jobs-with-nestjs-redis-3a4a53832ad0>

⁸ OpenRouter Vs AI Gateway: Differences, Use Cases & Best Choice

<https://www.truefoundry.com/blog/openrouter-vs-ai-gateway>

9 Vector embeddings | OpenAI API

<https://developers.openai.com/api/docs/guides/embeddings>

10 11 Queues | NestJS - A progressive Node.js framework

<https://docs.nestjs.com/techniques/queues>